Experiment 5: RNN for Text Classification

➢ Objective

Design and implement a deep learning network for classification of textual documents using LSTM. Understand how recurrent neural networks capture sequential dependencies in text data.

➢ Prerequisites

- Basics of Natural Language Processing (NLP)
- Understanding of RNN and LSTM architectures
- Familiarity with Keras/TensorFlow
- Knowledge of tokenization, padding, and label encoding

➢ Theory Background

Traditional neural networks struggle with sequential data like text, where word order and context matter. **Recurrent Neural Networks (RNNs)** address this by maintaining a hidden state that captures dependencies across time steps. However, standard RNNs face **vanishing gradient problems**, making them ineffective for long sequences.

Long Short-Term Memory (LSTM) networks overcome this by introducing **gates** (input, forget, output) that control the flow of information, allowing the model to capture long-range dependencies.

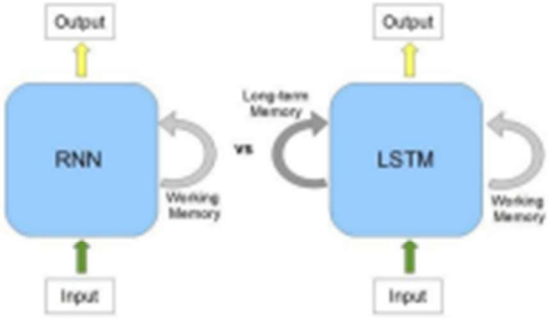


Image Credit: Abeyaantrix Edusoft

❖ What is LSTM?

LSTM (Long Short-Term Memory) is a type of neural network that can remember important information for long periods of time. Think of it like a smart notebook that can decide what to remember, what to forget, and what to share.

❖ The Main Idea

Imagine you're reading a long story. As you read:

- You remember important characters and plot points
- You forget irrelevant details
- You use what you remember to understand new parts of the story

LSTM works similarly with data sequences!

❖ The Memory Cell

At the heart of LSTM is a memory cell - like a storage box that holds important information. This cell is controlled by three "gates" (think of them as smart filters):

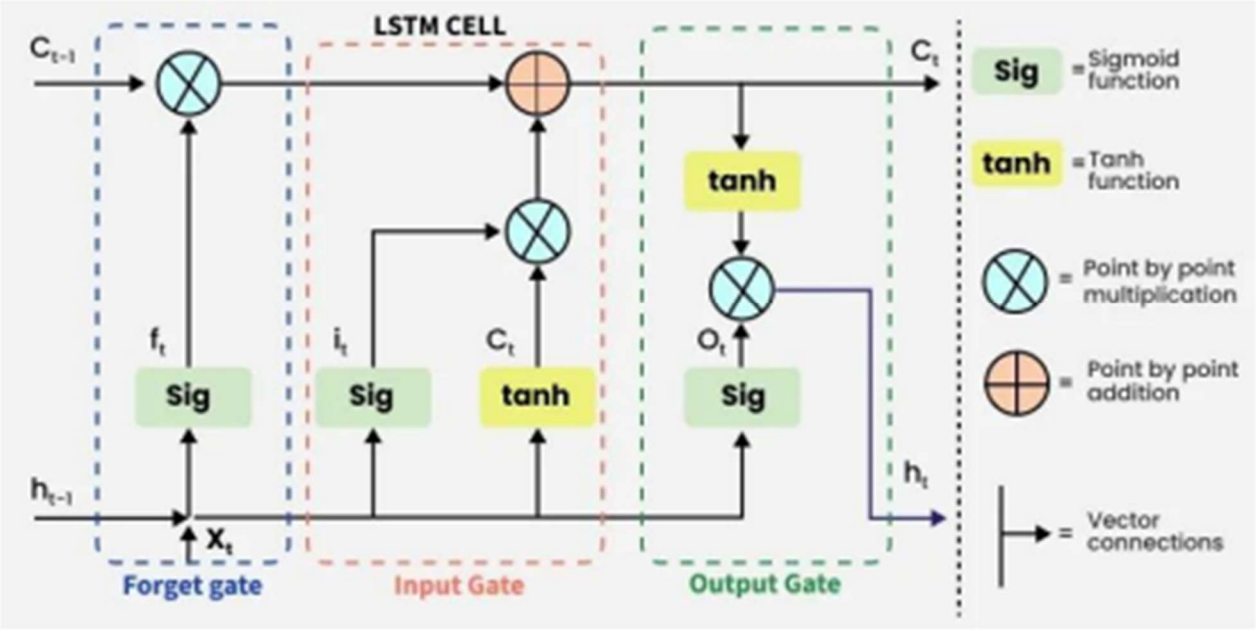


Image Credit: Abevaantrix Edusoft

- **The Three Gates**

1. 🚪 Forget Gate - "What should I forget?"

- Job: Decides what old information is no longer needed
- How it works: Looks at new input and previous memory, then decides what to throw away
- Example: When reading "The cat was black. The dog was brown.", it might forget the cat's color when focusing on the dog

2. 📥 Input Gate - "What new information should I remember?"

- Job: Decides what new information is worth storing
- How it works: Filters incoming information and decides what's important enough to remember
- Example: When you read "John is the main character", it decides this is important to remember

3. 📤 Output Gate - "What should I share right now?"

- Job: Decides what information from memory to use for the current output
- How it works: Looks at what's stored in memory and decides what's relevant for the current situation
- Example: When asked "Who is the main character?", it retrieves "John" from memory

❖ **How It All Works Together**

1. New information arrives (like a new word in a sentence)
2. Forget Gate says: "Do I need to forget anything old?"
3. Input Gate says: "Should I remember this new information?"
4. Memory gets updated with the new important stuff
5. Output Gate says: "What should I output based on what I remember?"
6. Output is produced and sent to the next step

❖ **Why is LSTM Special?**

- Regular neural networks have short memory (like goldfish)
- LSTM networks can remember things from much earlier in the sequence (like elephants)
- This makes them great for tasks like:
    - Language translation
    - Speech recognition
    - Stock price prediction
    - Any task where context from the past matters

❖ **Simple Analogy**

Think of LSTM like a smart assistant reading your emails:

- **Forget Gate:** "This old meeting info is outdated, I'll forget it"
- **Input Gate:** "This new project deadline is important, I'll remember it"
- **Output Gate:** "When asked about upcoming deadlines, I'll mention the project"

Why LSTM for Text Classification?

- Captures word order and semantic context
- Effective for sentiment analysis, topic classification, and document categorization
- Works with embeddings → learns meaningful feature representations of words

Choosing RNN Type:

- **Start with LSTM**: Good default choice
- **Try GRU**: If you need speed or have limited data
- **Use Bidirectional**: For better context understanding
- **Add Attention**: For interpretability and performance

Hyperparameter Guidelines:

- **Sequence Length**: Start with 200-500 for most text tasks
- **Embedding Dimension**: 128-300 typically works well
- **Hidden Units**: 64-256 depending on complexity
- **Dropout**: 0.2-0.5 to prevent overfitting

Common Pitfalls to Avoid:

1. **Too long sequences**: Leads to memory issues and slower training
2. **No dropout**: Model will overfit on training data
3. **Wrong padding**: Can confuse the model
4. **Ignoring class imbalance**: Use weighted loss or resampling
5. **No validation set**: Can't detect overfitting

➢ Implementation Steps

Step 1: Import Libraries

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

Step 2: Load and Preprocess Dataset

```
# Load dataset
data = pd.read_csv("/content/drive/MyDrive/sentiment.csv")
print(data.head())

# Encode labels
le = LabelEncoder()
data['Sentiment'] = le.fit_transform(data['Sentiment'])  # e.g.,
positive=2, negative=0, neutral=1

# Tokenize text
tokenizer = Tokenizer(num_words=5000, oov_token="<OOV>")
tokenizer.fit_on_texts(data['Sentence'])
sequences = tokenizer.texts_to_sequences(data['Sentence'])

# Pad sequences to fixed length
padded = pad_sequences(sequences, maxlen=50, padding='post')

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(padded,
data['Sentiment'], test_size=0.2, random_state=42)
```

Step 3: Build LSTM Model

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=5000, output_dim=16,
input_length=50),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(16, activation='relu'),
 tf.keras.layers.Dense(3, activation='softmax')  # 3 sentiment
classes
])
model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
```

Step 4: Train the Model

```
history = model.fit(X_train, y_train, epochs=5, batch_size=16,
validation_data=(X_test, y_test))
```

Step 5: Evaluate the Model

```
loss, acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {acc:.2f}")
```

Step 6: Sample Predictions

```
print(data[
print("Model predictions =", 'Sentence'][:2])
model.predict(padded[:2]))
print("Actual Sentiments =", data['Sentiment'][:2]
```

➢ Expected Results

- Training accuracy improves with epochs (~85–90% on sentiment dataset).
- Validation accuracy stabilizes around 80–85%.
- Predictions return probabilities across 3 sentiment classes.

Experiment 5: Text Classification (LSTM)

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load dataset
data = pd.read_csv("/content/drive/MyDrive/sentiment.csv")

# Encode labels
le = LabelEncoder()
data['Sentiment'] = le.fit_transform(data['Sentiment'])

# Tokenize & Pad
tokenizer = Tokenizer(num_words=5000, oov_token="<OOV>")
tokenizer.fit_on_texts(data['Sentence'])
sequences = tokenizer.texts_to_sequences(data['Sentence'])
padded = pad_sequences(sequences, maxlen=50, padding='post')

X_train, X_test, y_train, y_test = train_test_split(padded, data['Sentiment'], test_size=0.2, random_state=42)

# Build LSTM
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=5000, output_dim=16, input_length=50),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, batch_size=16, validation_data=(X_test, y_test))
print(model.evaluate(X_test, y_test))
```

➢ Viva Questions

1. What is the difference between RNN and LSTM?
2. Why do we use padding in text preprocessing?
3. Explain the role of word embeddings in text classification.
4. What are vanishing gradients, and how does LSTM solve them?
5. Compare Bag-of-Words vs LSTM approaches for text classification.
6. Why is softmax used in the output layer for multi-class classification?
7. How would performance change if we used GRU instead of LSTM?
8. Explain the importance of sequence length (maxlen) in padding.
9. How can we prevent overfitting in LSTM models?
10. Suggest real-world applications of LSTM-based text classification.